

UVM UVCs

(UVM Verification Components)

SyoSil

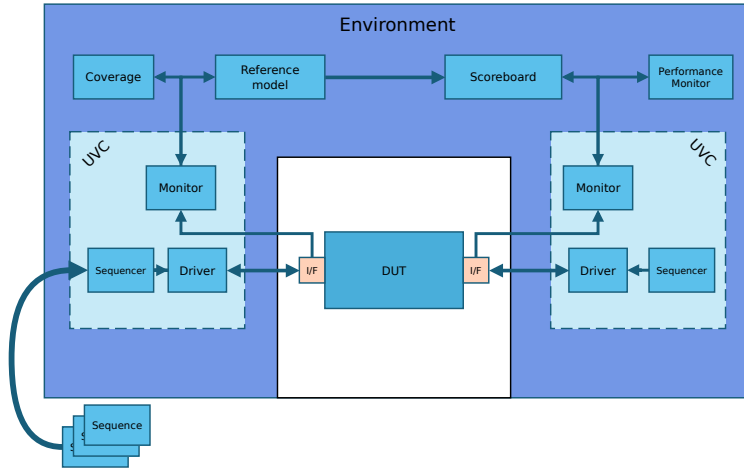
2026

Contents

- UVC Overview
- Interface
- Sequence Item
- Monitor
- Sequence, Sequencer and Driver
- Configuration Object
- Coverage
- Sequence Library
- Sequencer/Driver connection and handshake
- SSDT UVC Implementation

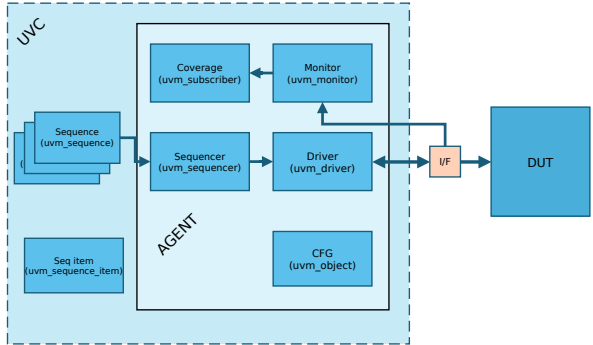
UVC Overview

UVM Verification Components (UVCs)



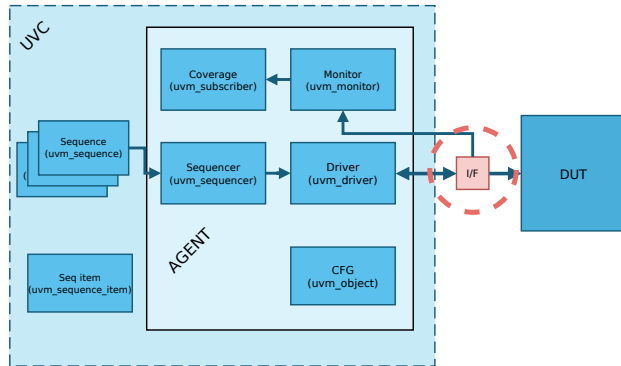
UVC

- At least following elements:
 - Agent (uvm_agent)
 - Interface
 - Sequence item (uvm_sequence_item)
 - Monitor (uvm_monitor)
 - Driver (uvm_driver)
 - Sequencer (uvm_sequencer)
- Usually also:
 - Configuration object
 - Sequence library
 - Coverage collection



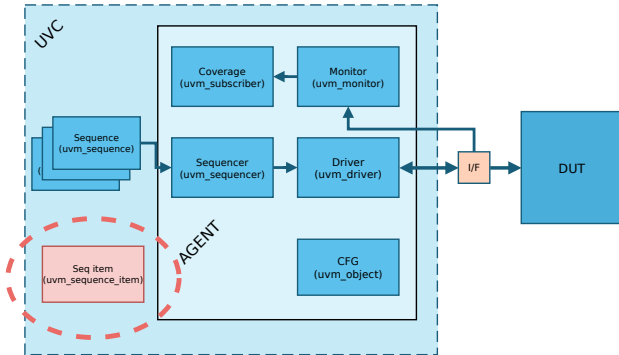
Interface

- Not in cocoTB or PyUVM
 - Is present in SystemVerilog
- Necessary for UVM to hook up to RTL DUT
- Already introduced in UVM lecture
- Contains
 - Wires for DUT pin hookup
 - Potentially helper code for the UVM driver



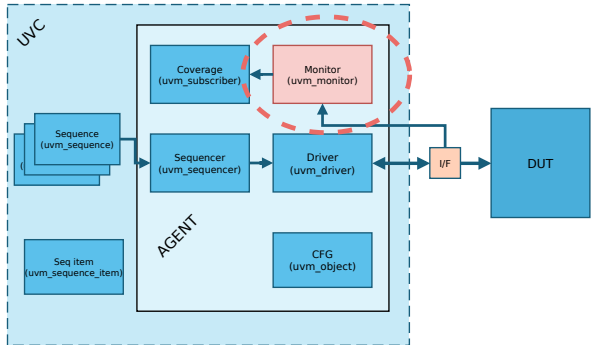
Sequence item (uvm_sequence_item)

- UVM Object
- Capturing a “transaction” on the abstract level
 - Rand data members
 - Constraints
- Used for (at least):
 - Stimuli: Can be randomized
 - Recorded traffic, observed by monitor
 - Coverage analytics
 - Reference Model/ Scoreboard



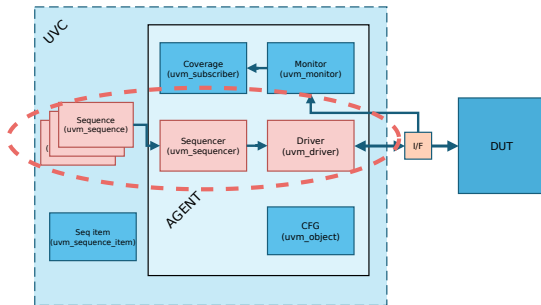
Monitor (uvm_monitor)

- UVM Component
- Monitors bus at pin level
- Publishes transactions for analysis
 - Coverage
 - Reference model & scoreboarding
- Responsible for validating bus protocol
 - Contains protocol checks
 - May also contain some pin coverage



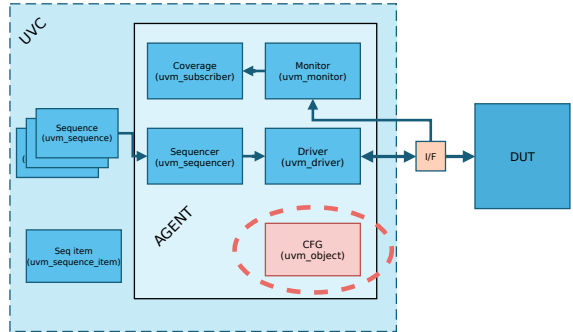
Sequences, Sequencer, Driver

- Sequences (`uvm_sequence`)
 - UVM Object
 - Sequences provide transactions
- Sequencer (`uvm_sequencer`)
 - UVM Component
 - Arbitrates between sequences requesting access to driver
 - Routes response to appropriate sequence
- Driver (`uvm_driver`)
 - UVM Component
 - Converts transactions to pin wiggles
 - May provide transaction response
 - Communicates with sequencer through function calls (backed by TLM channels)
 - Communicates with DUT through interface



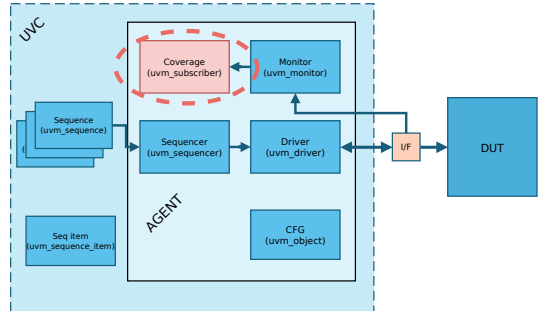
Configuration Object (uvm_object)

- UVM Object
- Contains
 - Virtual interface reference
 - UVC configuration options
 - Collects relevant configuration options into single object → fewer configDB lookups
- One configuration object instance per UVC instance



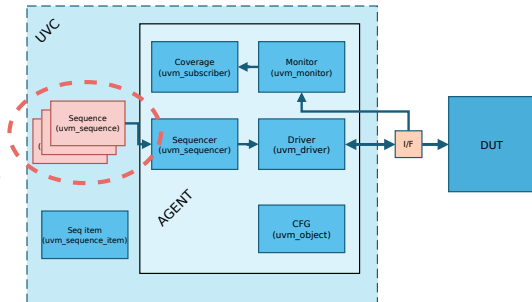
Coverage Collector (uvm_subscriber)

- UVM Component
- Subscribing to monitor observed sequence items
- Protocol centric coverage model
 - SV functional coverage
 - Examples:
 - ▶ Access types
 - ▶ Address map
 - ▶ Transaction Ids



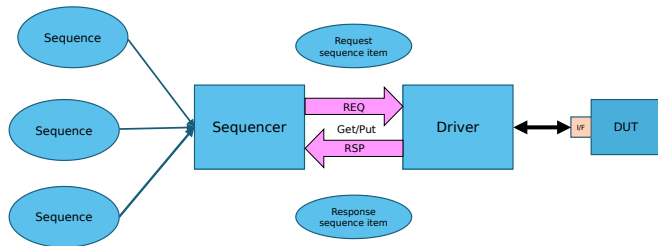
Sequence Library

- UVM Sequences
- UVC should provide basic (atomic) sequences, a library for reuse
 - E.g. single read, single write
- Testbench/testcase writer can extend, constrain, combine, re-use
- UVC sequence library should not implement higher level DUT specific protocol



Sequencer/Driver Connection and Handshake

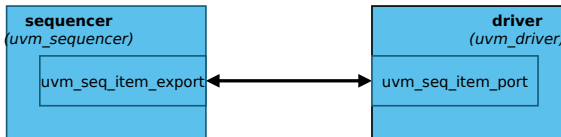
Overview



- Sequences generate sequence items (transactions)
- Sequencer arbitrates between active sequences
- Chosen sequence provides a sequence item for the driver
- Driver converts sequence item to pin wiggles
- Response is routed back to sequence

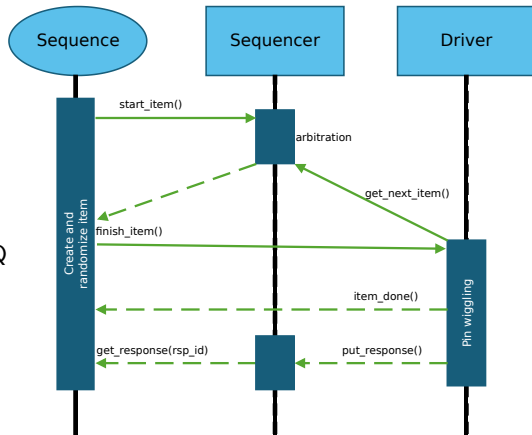
Sequencer (uvm_sequencer)/Driver (uvm_driver)

- The sequencer is derived from `uvm_sequencer`
- The driver is derived from `uvm_driver`
- The Driver and Sequencer connect through a bi-directional port connection



uvm_seq_item_port API

- `start_item()`
 - Waits until driver is ready for next seq item.
- `get_next_item()`
 - Gets REQ item from sequencer FIFO
- `finish_item()`
 - Sets item as ready for the driver; waits for driver to call `item_done()`.
- `item_done(rsp=None)`
 - Notifies sequencer that driver is done with REQ
 - Unblocks the `finish_item()` call in the providing sequence
- `put_response(item)`
 - Send the response item
- `get_response(rsp_id)`
 - Gets a specific response for ID
 - If no response is ready, then it will block

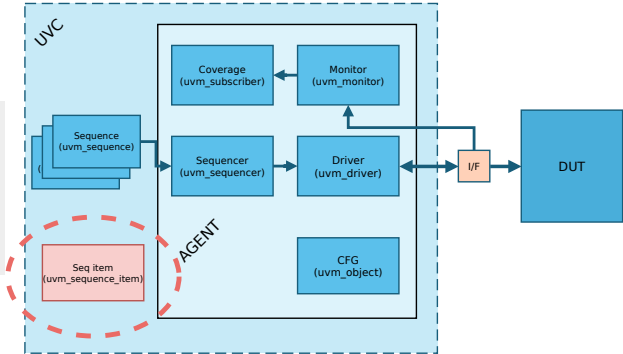


SSDT UVC Implementation

SSDT UVC: Sequence Item

```
@vsc.randobj
class uvc_ssdt_seq_item(uvm_sequence_item):
    def __init__(self, name):
        super().__init__(name)
        self.data = vsc.rand_bit_t(8)

    (...)
```



SSDT UVC: Sequences, Sequencer, Driver

```
class uvc_ssdt_basic_seq(uvc_ssdt_base_seq):
    async def body(self):
        await super().body()
        if self.seq_item is None:
            seq_item_name =
                self.sequencer.get_full_name() +
                "uvc_ssdt_seq_item"
            self.seq_item =
                uvc_ssdt_seq_item.create(seq_item_name)
        await self.start_item(self.seq_item)
        await self.finish_item(self.seq_item)
        self.rsp = await self.get_response()

class uvc_ssdt_default_seq(uvc_ssdt_base_seq):
    async def body(self):
        (...)
        await self.start_item(self.seq_item)
        if not self.seq_was_randomized:
            self.seq_item.randomize()
        await self.finish_item(self.seq_item)
        self.rsp = await self.get_response()
```

```
class uvc_ssdt_base_driver(uvm_driver):
    while True:
        self.req = await
        self.seq_item_port.get_next_item()
        self.rsp = self.req.clone()
        (...)
        await self.drive_transaction()
        self.seq_item_port.item_done()
        self.seq_item_port.put_response(self.rsp)

class
uvc_ssdt_producer_driver(uvc_ssdt_base_driver):
    (...)
    async def drive_transaction(self):
        self.vif.data.value = self.req.data
        self.vif.valid.value = 1
        self.rsp.data = self.req.data
        await RisingEdge(self.vif.clk)
```

SSDT UVC: Monitor

```
class uvc_ssdt_monitor(uvm_monitor):  
    async def run_phase(self):  
        await super().run_phase()  
        seq_item_counter = 0  
        while True:  
            await RisingEdge(self.vif.clk)  
            await ReadOnly()  
            if self.vif.valid.value == 1:  
                seq_item_name = self.get_full_name() +  
                    ".sdt_mon_item_" + str(seq_item_counter)  
                item = uvc_ssdt_seq_item.create(  
                    name=seq_item_name  
                )  
                seq_item_counter += 1  
  
                item.data = int(self.vif.data.value)  
                self.mon_analysis_port.write(item)
```

